Kernel testing frameworks

Kernel Recipes
September 21st, 2026

Shuah Khan
Kernel Maintainer & Linux Fellow
The Linux Foundation
LinkedIn - shuah-khan
skhan@linuxfoundation.org



We can all agree that it is important to test the kernel. There are a number of tools for code coverage, dynamic and static analysis to choose from to test the kernel. Linux kernel provides two testing frameworks to write and run tests. A majority of kernel tests are written using these two frameworks. Having a rich set of tools and frameworks is great. However it can be challenging to know which one to chose.

In this talk, Shuah will give an overview of tools and frameworks and their differences, and how to use them to achieve the your kernel testing objectives and goals.

Humble beginnings



Before kselftest and KUnit

- Relied on external testing, e.g: Linux Testing Project
- Tests lagged kernel releases
- Test developemnet was an afterthought
- Handful of tests in tools, testing, and selftests directories
- Features and API went in without tests and regression tests
- Goal: be able to test kernel with a single command from the main level
- Encourage developers to submit features and tests together
- Developers help themselves by writing tests

Fast forward 15+ years

- External testing is still relevant e.g: Linux Testing Project
- Tests are included in kernel releases
- Writing tests is no loger an afterthought
- Kselftest since 2014 and KUnit since 2018
- 100s of tests in tools, testing, and selftests directories
- Features and API go in with tests and regression tests
- Test kernel with a single command from the main level
- Developers submit features and tests together
- Test rings run selftests (kselftest and KUnit)

Why kselftest?

- Regression test suite
- Openbox tests
- Tests kernel from user-space
- User-space applications
  - (Shell scripts, C programs)
  - Kernel Test modules used to exercise kernel code paths

- Allows for breadth and depth coverage (error paths etc.)
- Not for workload or application testing

Why kselftest?

- Perfect for
  - feature, functional and regression testing
  - bug fix focused regression testing and subsystem testing
  - testing user APIs, system calls, critical user paths, common use cases
  - end to end regression testing

- Open and Closed box testing – quick assurance that everything works

Kernel Validation With Kselftest

Why KUnit?

- Focuses on in-kernel testing
- Perfect for:
  - testing internal kernel APIs
  - libraries, drivers, …,
  - individual *units* of code
- Perfect for unit testing
  - makes it easy to test all the edge cases
- Closed box test suites

KUnit Testing Strategies

Kselftest & KUnit

Kselftest

- Good for depth testing covering deeper code paths
- Good for testing commonly used code paths
- A good test could test some error paths
- Multiarch support

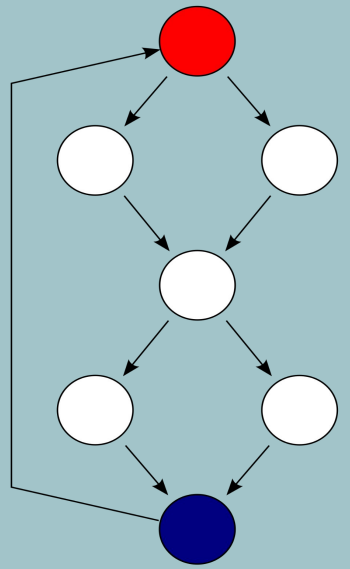
KUnit

- Good for targeting error paths & edge cases
- Easier and faster for zeroing in on a kernel area
- Does KUnit support running on architectures other than UML?
  - Yes.

Cyclomatic complexity by McCabe - Wikipedia

```
if (condition1() )
        function1();
else
        function2();

if (condition2() )
        function3();
else
        function4();
```



JulesH, Public domain, via Wikimedia Commons

Cyclomatic complexity by McCabe - Wikipedia

- McCabe's complexity is a measure of the number of states, or branches a function can achieve
- Testing all edge cases?
  - Can a closed box test reach an arbitrary edge case in the kernel from a syscall?
  - Reaching every state is not easily achievable
- Solution: Call functions directly to test edge cases
- KUnit is a really practical way to test the vast majority of edge cases.

Kernel Testing Guide

- Writing and Running Tests
- Kselftest vs. KUnit
- Which one to choose?
- How to choose?

Choosing a framework

- What are the goals?
- Regression testing?
- Closed or open box
- Are there any user-space interaction?
- Is it a unit test?
- What about multi-arch support?
- What about code coverage?

How many tests (as of 7.3-rc3)?

- Kselftest
  - 216 individual Makefiles (up from 192 in June 2025
  - 125 subsystems (directories)
  - Run default suite:
    - make kselftest
  - Build
    - make kselftest-all
  - Build and install
    - make kselftest-install
  - Running a subset of tests
    - make TARGETS="size timers" kselftest
  - Direct run
    - cd tools/testing/selftests/mm; make run_tests
    - make -C tools/testing/selftests/mm run_tests

How many tests (as of 7.3-rc3)?

- KUnit
  - Spread around the kernel
  - ./tools/testing/kunit/kunit.py run
    - Testing complete. Ran 876 tests: passed: 849, skipped: 27
    - Elapsed time: 117.089s total, 6.241s configuring, 64.548s building, 46.276s running
  - ./tools/testing/kunit/kunit.py run --alltests
    - Testing complete. Ran 9286 tests: passed: 9234, skipped: 52
    - Elapsed time: 222.917s total, 2.506s configuring, 142.276s building, 78.069s running

My kselftest pull request testing workflow

- Changes to a few tests
  - build and run changed tests
- Framework changes
  - make kselftest-all,
  - make kselftest
  - make ksleftest-install
- Arch build in some cases

My KUnit pull request testing workflow

- Build tests on kselftest and linux-next
  - Enable CONFIG_KUNIT and CONFIG_KUNIT_ALL_TESTS
  - make allmodconfig; make
- Default run test - um
  - ./tools/testing/kunit/kunit.py run
  - ./tools/testing/kunit/kunit.py run --alltests
- X86_64 run test
  - ./tools/testing/kunit/kunit.py run --arch x86_64
  - ./tools/testing/kunit/kunit.py run --alltests --arch x86_64

Frequently asked questions

- Is there support for expected fail/pass
  - Yes - References:
  - Linux Kernel Selftests, kselftes.h, kselftest_harness.h
- What happens when build dependencies aren't met?
  - Build as many test cases as they can
  - Goal: Continue when a test fails to build
    - Override with FORCE_TARGETS=1 for builds to fail when a test fails to build
-
- What happens when dependencies aren't met?
  - Tests run as many test cases as they can
  - Skipping tests is fine grain. Skip just the ones with unmet dependencies: config, feature, etc.
  - Default kselftest run attempts to run all tests skipping the ones with unmet dependencies.

What’s next?

- How do I know which tests to run for my use-case?
  - Kselftests or KUnit - both
- But which tests do I choose to run?

KCOV: code coverage

- KCOV keeps track of code run during execution
- CONFIG_ARCH_HAS_KCOV=y
- CONFIG_KCOV
- My tool to do a quick check of individual test coverage
  - kcov_collect [-c command] | addr2line -afi -e vmlinux

GCOV: code coverage

- GCOV keeps track of code run during execution
- CONFIG_GCOV_KERNEL
- CONFIG_ARCH_HAS_GCOV_PROFILE_ALL=y
- Generates reports
- Show what code ran, and what code did not

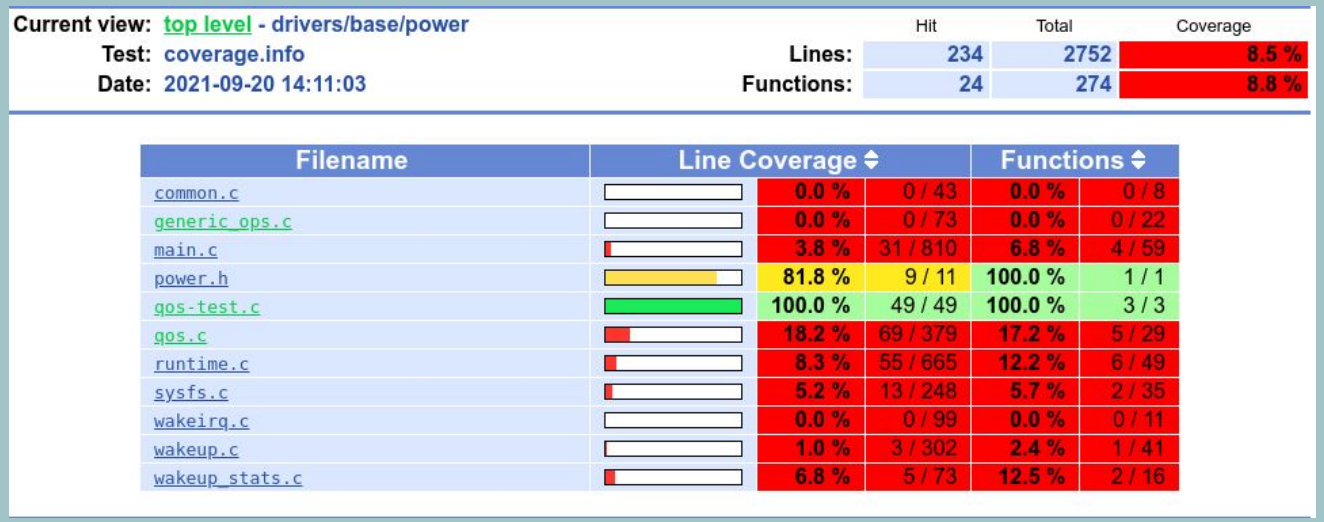
GCOV content from Brendan Higgins, KUnit maintainer

GCOV: How to coverage

```
150          1 : static int apply_constraint(struct dev_pm_qos_request *req,
151            :                               enum pm_qos_req_action action, s32 value)
152            : {
153          1 :         struct dev_pm_qos *qos = req->dev->power.qos;
154            :         int ret;
155            :
156          1 :         switch(req->type) {
157          1 :         case DEV_PM_QOS_RESUME_LATENCY:
158          1 :                 if (WARN_ON(action != PM_QOS_REMOVE_REQ && value < 0))
159          0 :                         value = 0;
160            :
161          1 :                 ret = pm_qos_update_target(&qos->resume_latency,
162            :                                            &req->data.pnode, action, value);
163          1 :                 break;
164          0 :         case DEV_PM_QOS_LATENCY_TOLERANCE:
165          0 :                 ret = pm_qos_update_target(&qos->latency_tolerance,
166            :                                            &req->data.pnode, action, value);
167          0 :                 if (ret) {
168          0 :                         value = pm_qos_read_value(&qos->latency_tolerance);
169          0 :                         req->dev->power.set_latency_tolerance(req->dev, value);
170            :                 }
171            :                 break;
172          0 :         case DEV_PM_QOS_MIN_FREQUENCY:
173            :         case DEV_PM_QOS_MAX_FREQUENCY:
174          0 :                 ret = freq_qos_apply(&req->data.freq, action, value);
175          0 :                 break;
176          0 :         case DEV_PM_QOS_FLAGS:
177          0 :                 ret = pm_qos_update_flags(&qos->flags, &req->data.flr,
178            :                                           action, value);
179          0 :                 break;
180            :         default:
181            :                 ret = -EINVAL;
182            :         }
183            :
184          1 :         return ret;
185            : }
```

GCOV: How to coverage

- Shows directory level summaries

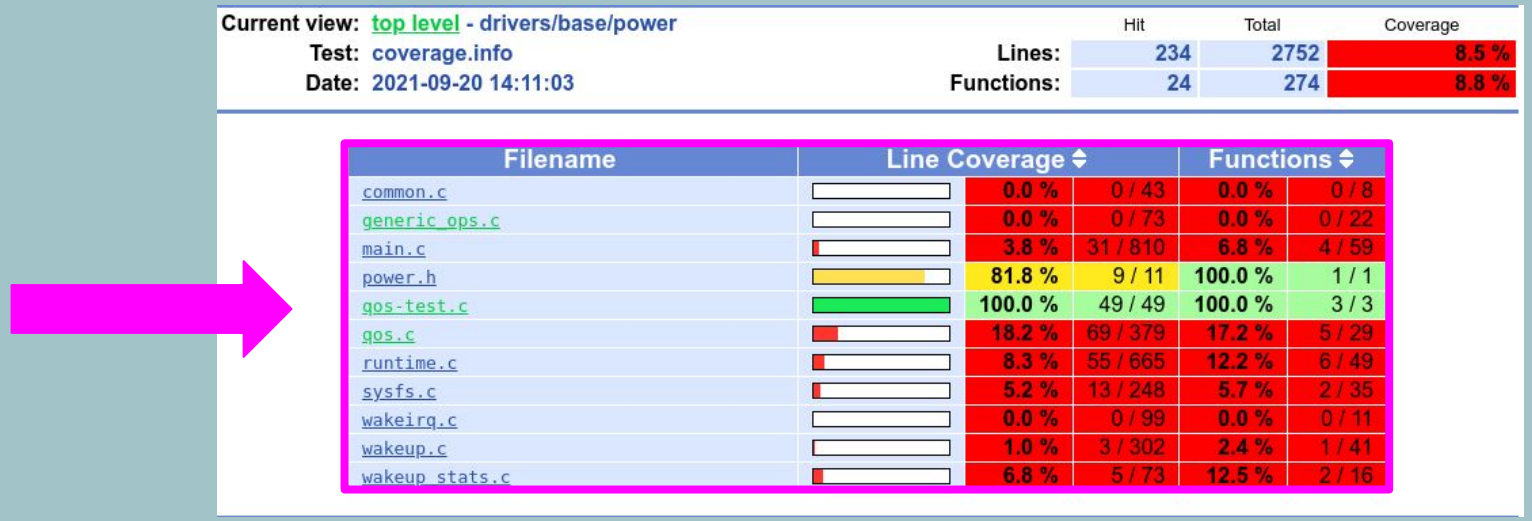


Current view:	top level - drivers/base/power		Hit	Total	Coverage
Test:	coverage.info	Lines:	234	2752	8.5 %
Date:	2021-09-20 14:11:03	Functions:	24	274	8.8 %

Filename	Line Coverage		Functions	
common.c	0.0 %	0 / 43	0.0 %	0 / 8
generic_ops.c	0.0 %	0 / 73	0.0 %	0 / 22
main.c	3.8 %	31 / 810	6.8 %	4 / 59
power.h	81.8 %	9 / 11	100.0 %	1 / 1
qos-test.c	100.0 %	49 / 49	100.0 %	3 / 3
qos.c	18.2 %	69 / 379	17.2 %	5 / 29
runtime.c	8.3 %	55 / 665	12.2 %	6 / 49
sysfs.c	5.2 %	13 / 248	5.7 %	2 / 35
wakeirq.c	0.0 %	0 / 99	0.0 %	0 / 11
wakeup.c	1.0 %	3 / 302	2.4 %	1 / 41
wakeup_stats.c	6.8 %	5 / 73	12.5 %	2 / 16

GCOV: How to coverage

- Shows directory level summaries



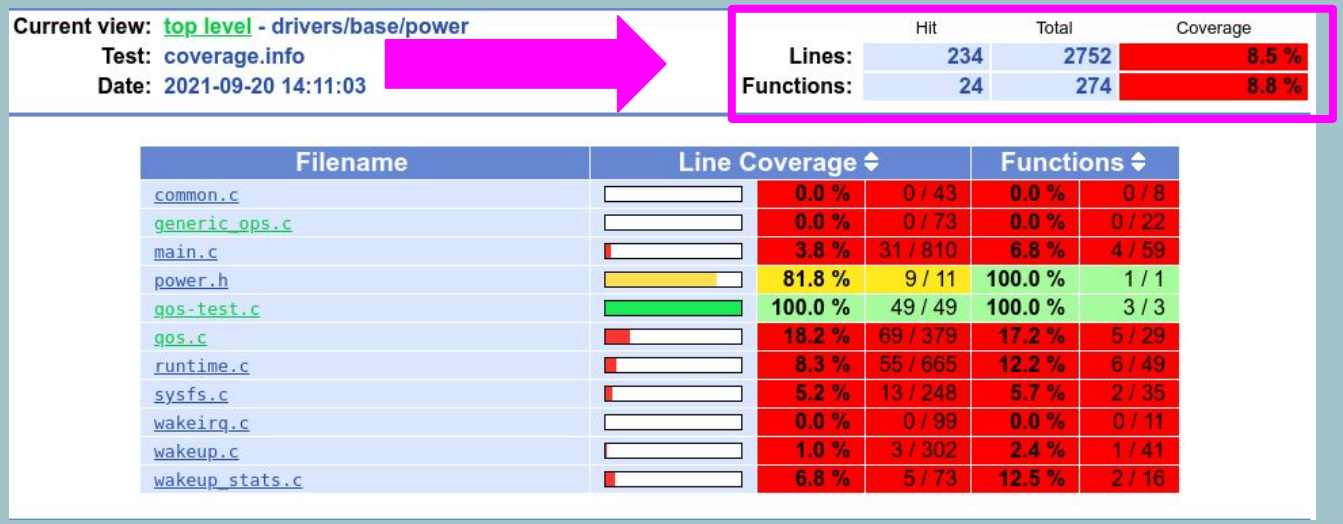
Current view: top level - drivers/base/power
Test: coverage.info
Date: 2021-09-20 14:11:03

	Hit	Total	Coverage
Lines:	234	2752	8.5 %
Functions:	24	274	8.8 %

Filename	Line Coverage		Functions	
common.c	0.0 %	0 / 43	0.0 %	0 / 8
generic_ops.c	0.0 %	0 / 73	0.0 %	0 / 22
main.c	3.8 %	31 / 810	6.8 %	4 / 59
power.h	81.8 %	9 / 11	100.0 %	1 / 1
qos-test.c	100.0 %	49 / 49	100.0 %	3 / 3
qos.c	18.2 %	69 / 379	17.2 %	5 / 29
runtime.c	8.3 %	55 / 665	12.2 %	6 / 49
sysfs.c	5.2 %	13 / 248	5.7 %	2 / 35
wakeirq.c	0.0 %	0 / 99	0.0 %	0 / 11
wakeup.c	1.0 %	3 / 302	2.4 %	1 / 41
wakeup_stats.c	6.8 %	5 / 73	12.5 %	2 / 16

GCOV: How to coverage

- Shows directory level summaries
- Shows overall summary as coverage number



Current view: top level - drivers/base/power
Test: coverage.info
Date: 2021-09-20 14:11:03

	Hit	Total	Coverage
Lines:	234	2752	8.5 %
Functions:	24	274	8.8 %

Filename	Line Coverage		Functions	
common.c	0.0 %	0 / 43	0.0 %	0 / 8
generic_ops.c	0.0 %	0 / 73	0.0 %	0 / 22
main.c	3.8 %	31 / 810	6.8 %	4 / 59
power.h	81.8 %	9 / 11	100.0 %	1 / 1
qos-test.c	100.0 %	49 / 49	100.0 %	3 / 3
qos.c	18.2 %	69 / 379	17.2 %	5 / 29
runtime.c	8.3 %	55 / 665	12.2 %	6 / 49
sysfs.c	5.2 %	13 / 248	5.7 %	2 / 35
wakeirq.c	0.0 %	0 / 99	0.0 %	0 / 11
wakeup.c	1.0 %	3 / 302	2.4 %	1 / 41
wakeup_stats.c	6.8 %	5 / 73	12.5 %	2 / 16

Code Coverage **IS**

- A great way to quickly find what code **IS** tested and what code **IS NOT** tested.
- Allows you to quickly identify problem areas, and drill down into a report.
- Identify missed branches.
- Identify unused code.

Code Coverage **IS**: Example

- Imagine we are testing some code:

```
static int __dev_pm_qos_add_request(struct device *dev,
                                    struct dev_pm_qos_request *req,
                                    enum dev_pm_qos_req_type type, s32 value)
{
        int ret = 0;

        if (!dev || !req || dev_pm_qos_invalid_req_type(dev, type))
                return -EINVAL;

        if (WARN(dev_pm_qos_request_active(req),
                 "%s() called for already added request\n", __func__))
                return -EINVAL;

        if (IS_ERR(dev->power.qos))
                ret = -ENODEV;
        else if (!dev->power.qos)
                ret = dev_pm_qos_constraints_allocate(dev);

        trace_dev_pm_qos_add_request(dev_name(dev), type, value);
        if (ret)
                return ret;

        req->dev = dev;
        req->type = type;
        if (req->type == DEV_PM_QOS_MIN_FREQUENCY)
                ret = freq_qos_add_request(&dev->power.qos->freq,
                                           &req->data.freq,
                                           FREQ_QOS_MIN, value);
        else if (req->type == DEV_PM_QOS_MAX_FREQUENCY)
                ret = freq_qos_add_request(&dev->power.qos->freq,
                                           &req->data.freq,
                                           FREQ_QOS_MAX, value);
        else
                ret = apply_constraint(req, PM_QOS_ADD_REQ, value);

        return ret;
}
```

Code Coverage **IS**: Example

- Imagine we are testing some code:
- We can see that we have edge cases for
  - DEV_PM_QOS_MIN_FREQUENCY
  - DEV_PM_QOS_MAX_FREQUENCY
-

```
static int __dev_pm_qos_add_request(struct device *dev,
                                    struct dev_pm_qos_request *req,
                                    enum dev_pm_qos_req_type type, s32 value)
{
        int ret = 0;

        if (!dev || !req || dev_pm_qos_invalid_req_type(dev, type))
                return -EINVAL;

        if (WARN(dev_pm_qos_request_active(req),
                 "%s() called for already added request\n", __func__))
                return -EINVAL;

        if (IS_ERR(dev->power.qos))
                ret = -ENODEV;
        else if (!dev->power.qos)
                ret = dev_pm_qos_constraints_allocate(dev);

        trace_dev_pm_qos_add_request(dev_name(dev), type, value);
        if (ret)
                return ret;

        req->dev = dev;
        req->type = type;
        if (req->type == DEV_PM_QOS_MIN_FREQUENCY)
                ret = freq_qos_add_request(&dev->power.qos->freq,
                                           &req->data.freq,
                                           FREQ_QOS_MIN, value);
        else if (req->type == DEV_PM_QOS_MAX_FREQUENCY)
                ret = freq_qos_add_request(&dev->power.qos->freq,
                                           &req->data.freq,
                                           FREQ_QOS_MAX, value);
        else
                ret = apply_constraint(req, PM_QOS_ADD_REQ, value);

        return ret;
}
```

Code Coverage **IS**: Example

- Imagine we are testing some code:
- We can see that we have edge cases for
  - `DEV_PM_QOS_MIN_FREQUENCY`
  - `DEV_PM_QOS_MAX_FREQUENCY`
- The report shows us that our tests do not cover these edge cases.



```
static int __dev_pm_qos_add_request(struct device *dev,
                                    struct dev_pm_qos_request *req,
                                    enum dev_pm_qos_req_type type, s32 value)
{
        int ret = 0;

        if (!dev || !req || dev_pm_qos_invalid_req_type(dev, type))
                return -EINVAL;

        if (WARN(dev_pm_qos_request_active(req),
                 "%s() called for already added request\n", __func__))
                return -EINVAL;

        if (IS_ERR(dev->power.qos))
                ret = -ENODEV;
        else if (!dev->power.qos)
                ret = dev_pm_qos_constraints_allocate(dev);

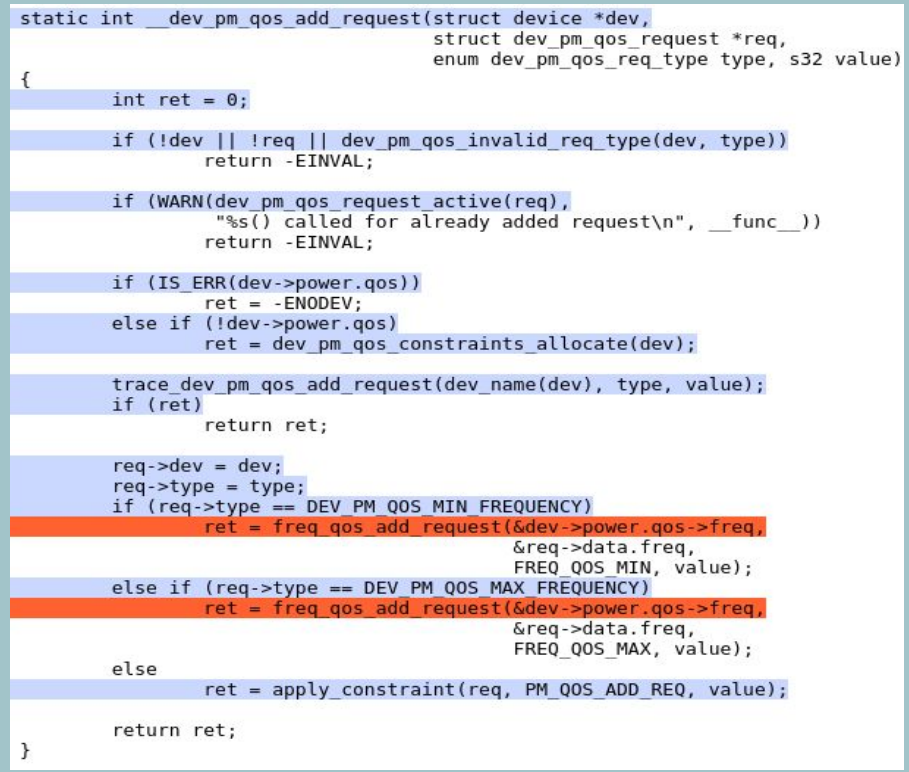
        trace_dev_pm_qos_add_request(dev_name(dev), type, value);
        if (ret)
                return ret;

        req->dev = dev;
        req->type = type;
        if (req->type == DEV_PM_QOS_MIN_FREQUENCY)
                ret = freq_qos_add_request(&dev->power.qos->freq,
                                           &req->data.freq,
                                           FREQ_QOS_MIN, value);
        else if (req->type == DEV_PM_QOS_MAX_FREQUENCY)
                ret = freq_qos_add_request(&dev->power.qos->freq,
                                           &req->data.freq,
                                           FREQ_QOS_MAX, value);
        else
                ret = apply_constraint(req, PM_QOS_ADD_REQ, value);

        return ret;
}
```

Code Coverage **IS**: Example

- Imagine we are testing some code:
- We can see that we have edge cases for
  - DEV_PM_QOS_MIN_FREQUENCY
  - DEV_PM_QOS_MAX_FREQUENCY
- The report shows us that our tests do not cover these edge cases.
- This shows the power of KUnit with coverage
  - We can (and do) call this function directly in tests



```
static int __dev_pm_qos_add_request(struct device *dev,
                                    struct dev_pm_qos_request *req,
                                    enum dev_pm_qos_req_type type, s32 value)
{
        int ret = 0;

        if (!dev || !req || dev_pm_qos_invalid_req_type(dev, type))
                return -EINVAL;

        if (WARN(dev_pm_qos_request_active(req),
                 "%s() called for already added request\n", __func__))
                return -EINVAL;

        if (IS_ERR(dev->power.qos))
                ret = -ENODEV;
        else if (!dev->power.qos)
                ret = dev_pm_qos_constraints_allocate(dev);

        trace_dev_pm_qos_add_request(dev_name(dev), type, value);
        if (ret)
                return ret;

        req->dev = dev;
        req->type = type;
        if (req->type == DEV_PM_QOS_MIN_FREQUENCY)
                ret = freq_qos_add_request(&dev->power.qos->freq,
                                           &req->data.freq,
                                           FREQ_QOS_MIN, value);
        else if (req->type == DEV_PM_QOS_MAX_FREQUENCY)
                ret = freq_qos_add_request(&dev->power.qos->freq,
                                           &req->data.freq,
                                           FREQ_QOS_MAX, value);
        else
                ret = apply_constraint(req, PM_QOS_ADD_REQ, value);

        return ret;
}
```

Code Coverage **IS NOT**

- Code coverage is a tool, not a panacea
- Code coverage helps quickly identify and prioritize problem areas
- Code coverage summaries **do not tell you whether your testing is good or bad**
  - What is the right amount of line coverage?
  - 50%?
  - 70%?
  - 90%?
  - 100%?

What’s the right coverage?

- How do we measure coverage?
  - % of lines?
  - % of functions?
  - % of branches?
- What about absolute vs incremental?

What’s the right coverage?

- Absolute coverage:
  - What you expect.
  - Everything in the entire codebase at some point in time.
- Incremental coverage:
  - The test coverage of the Δ in a change

Absolute vs. Incremental Coverage

- Incremental Coverage is usually more interesting
  - It’s much easier to achieve high incremental coverage immediately
  - Helps prioritize code more likely to be buggy
  - More actionable by developers
  - Code that has not changed in a long time is *more* likely to be fine

Absolute vs. Incremental Coverage

- Absolute Coverage is still important, just less important
  - Old code may be less likely to contain bugs…
  - ...but it’s often worse when it does
- Often easier for comparing coverage health of subsystems
- Easier to compute

Discussion – Q&A

Thank you for your time

